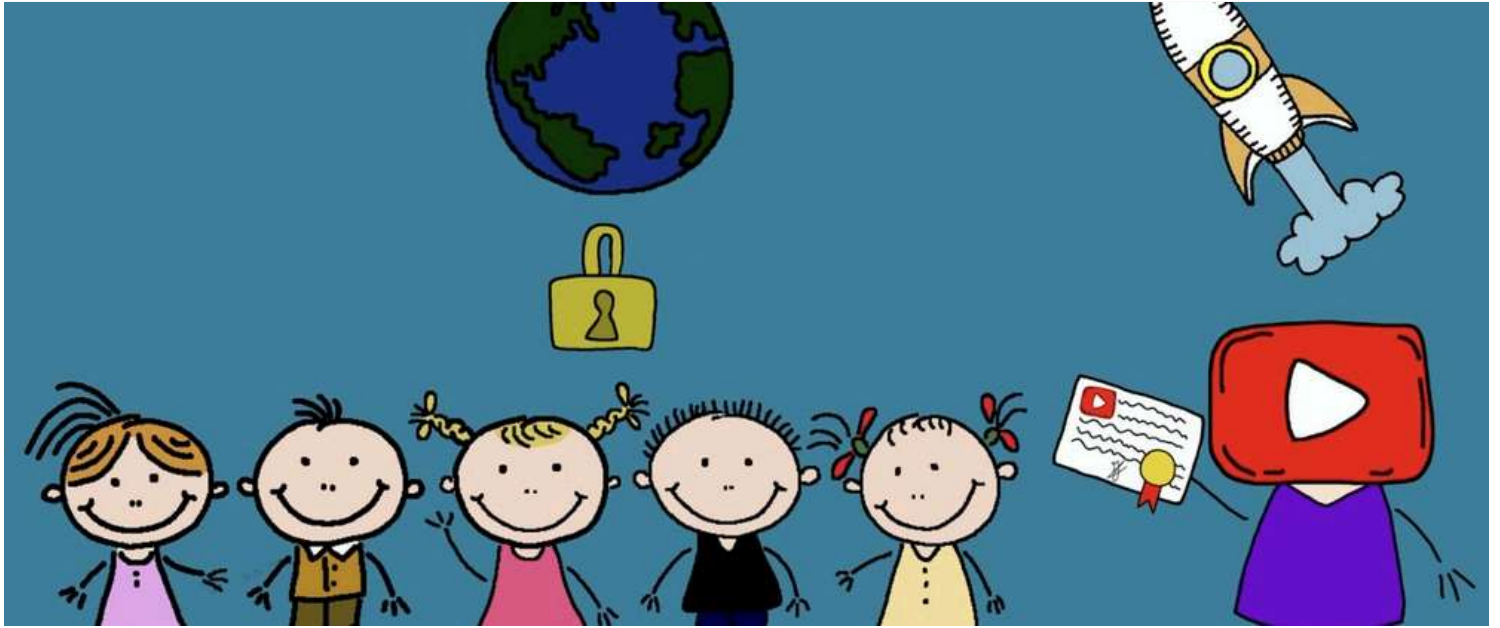




Jean-Paul Rustom

Posted on Aug 8, 2023 • Edited on Aug 12, 2023



9

TLS Certificates - Simplified for web developers

[#tls](#) [#webdev](#) [#react](#) [#node](#)

Please have a time to check the animated version ;)

Digital Certificates - Oversimplified For Web Developers



Introduction

Let's say JayP wants to connect to youtube.com over the public internet. Before any communication occurs, some concerns will arise:

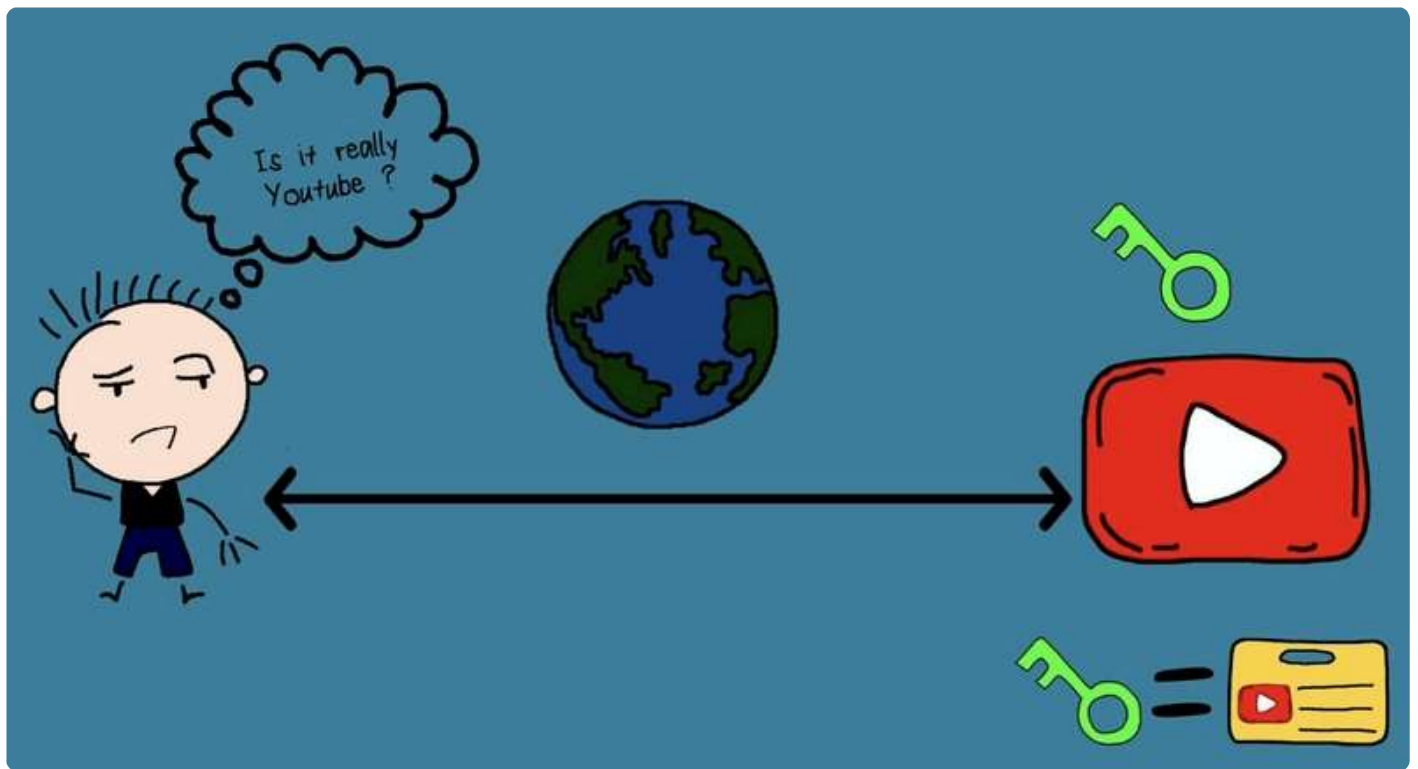
How can he verify that he is indeed communicating with youtube ? (Authentication)

How can JayP know that no one has intercepted his message ? (Confidentiality)

Also, how can he be sure that no one altered the message? (Integrity)

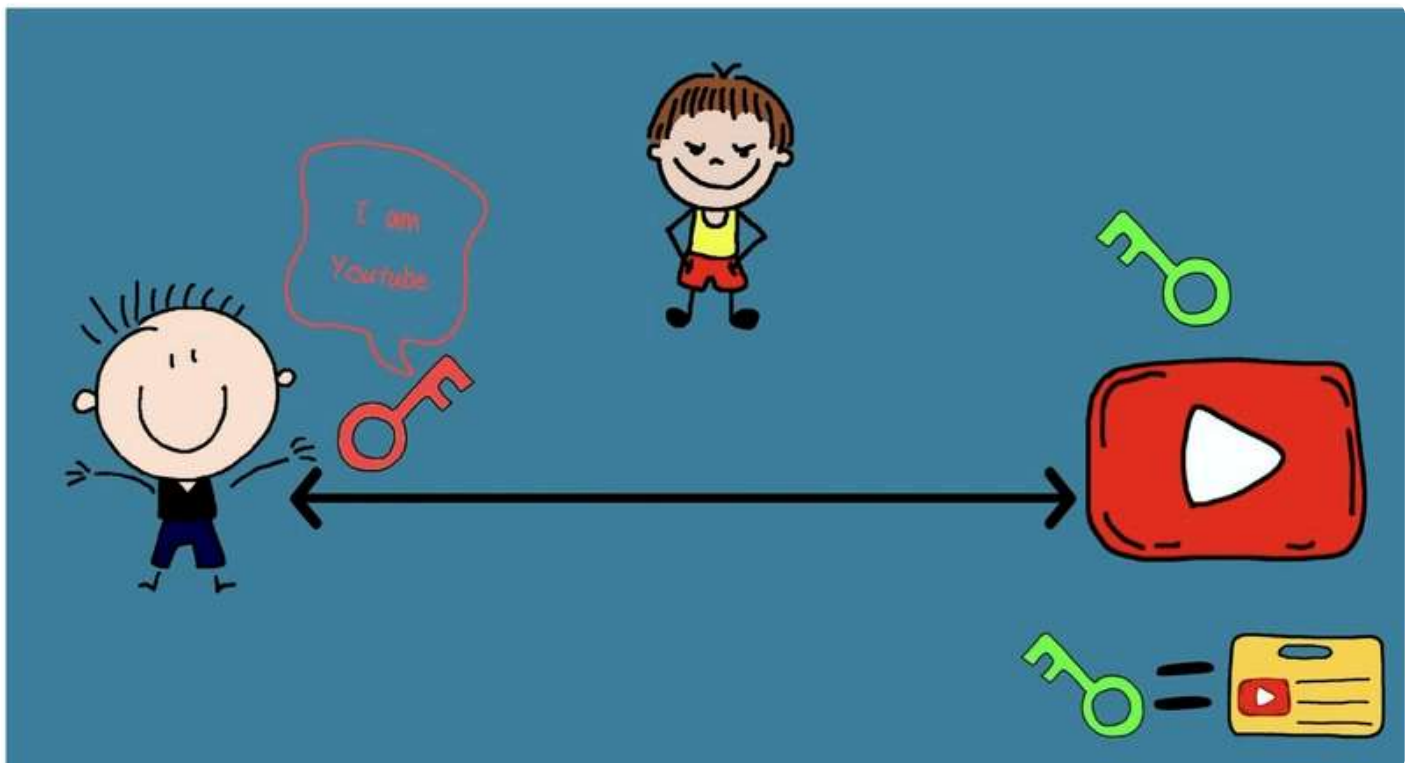
Well, let's tackle each problem separately.

First of all, JayP should verify the identity of Youtube. For that, he needs Youtube's public key. The public key represents youtube's identity.



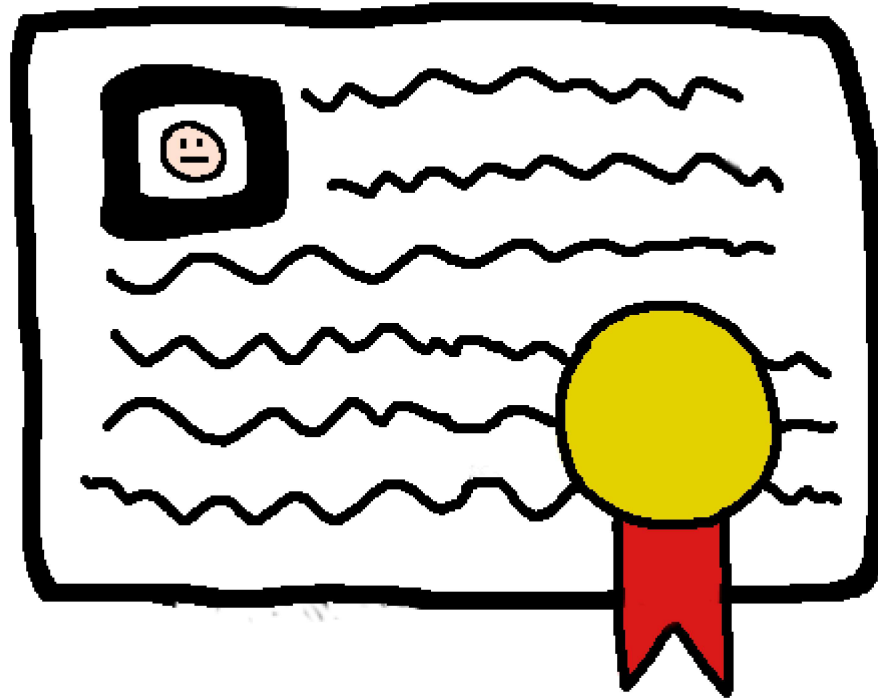
The Problem

The problem is, attackers have come up with ways to intercept a request from one computer to another computer on the Internet and launch a man in the middle attack. With this kind of attack, our attacker Chady would inject his own public key and have JayP think he is communicating with Youtube, while actually he is communicating with Chady.



Now you may think, scary, isn't it? Imagine what could be possible with this kind of attack, from stealing personal information such as login credentials or credit card details, to malware distribution or other malicious goals.

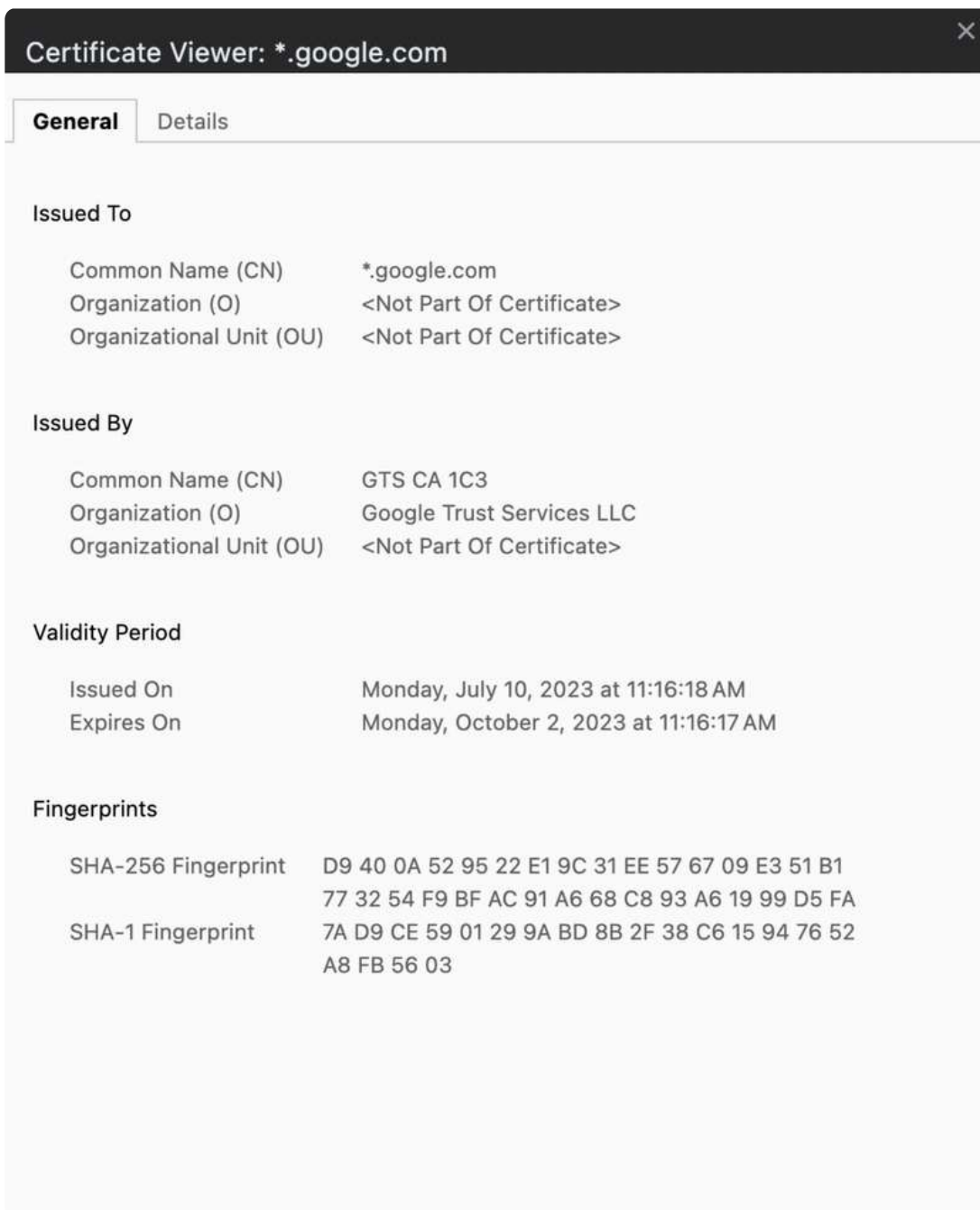
How then, can JayP be sure that the public key he received comes really from Youtube? That's where digital certificates come in. (TADAAA)



They ensure that the public key is indeed coming from Youtube, and not from a malicious Chady. Actually, digital certificates can be used not only for HTTPS, but also for mails, IOT, and VPN...

Information Inside TLS Certificate

A digital certificate will contain many information, let's explore the most important ones.



- **Issuer** is the third party which signed this certificate, we'll get into it in a bit. Some basic information will include the Common Name, the organization and the country.
- Of course, a digital certificate will also include its **Validity**, with two critical fields, not before and not after
- The **Subject** is the owner of the certificate and would be Youtube in our case. The subject will contain information such as common name, address, and most importantly, the public key.
- Lastly, a digital certificate include its **Signature**. This is the signature signed by the issuer that proves that this certificate is authentic.

Certificate Signature Algorithm
Certificate Signature Value
▼ Fingerprints
SHA-256 Fingerprint
SHA-1 Fingerprint

Field Value

23 FD 34 A1 ED F0 1D D5 C1 90 B1 85 F6 AF 84 8E
9D F4 99 0A 8B E0 B9 B5 31 B7 3D A5 4B 41 1B 9B
BC 4D AE 86 2B A1 74 D5 D4 6C C6 7E 9B 4F F6 C6
66 33 4C 69 22 DA 80 4F 31 8A E1 0A C8 30 D4 8B
B3 4C E3 80 15 5C A6 5B C9 05 3A 44 D3 F0 8D 2F
8B E7 9F CF 2F 76 0A 34 8A DC B6 E7 E3 49 8E 84

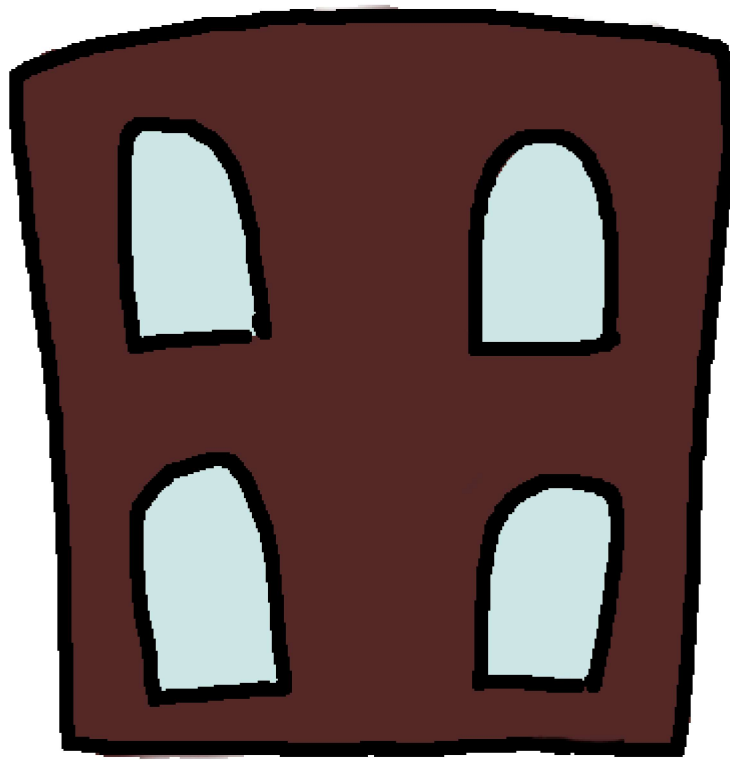
Export...

Who signs the digital certificate ?

Digital certificates are signed by entities called certificate authorities (for example DigiCert, Comodo, Symantec, Google Trust Services).

But wait, what are certificate authorities?

Well, in a nutshell, Certificate Authorities are trusted third-party organizations responsible for issuing digital certificates.

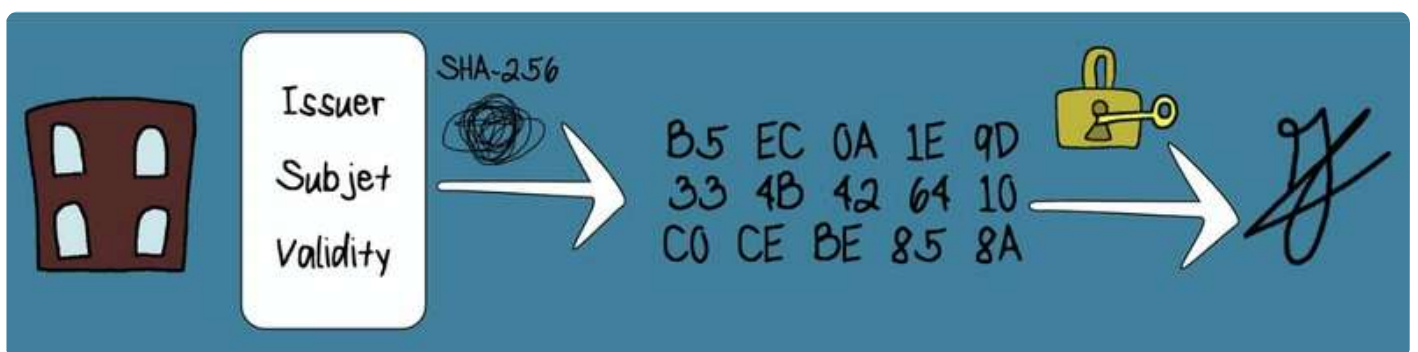


Certificate authorities have their own public and private keys. From that, let's explain how signatures are made.

The previously mentioned information contained in a digital certificate will be hashed using for eg SHA-256 algorithm.

After the hash is generated, the certificate authority encrypts the hash using its private key using RSA asymmetric key encryption. This encrypted hash is the signature of a certificate, and can only be decrypted using the certificate authority's public key.

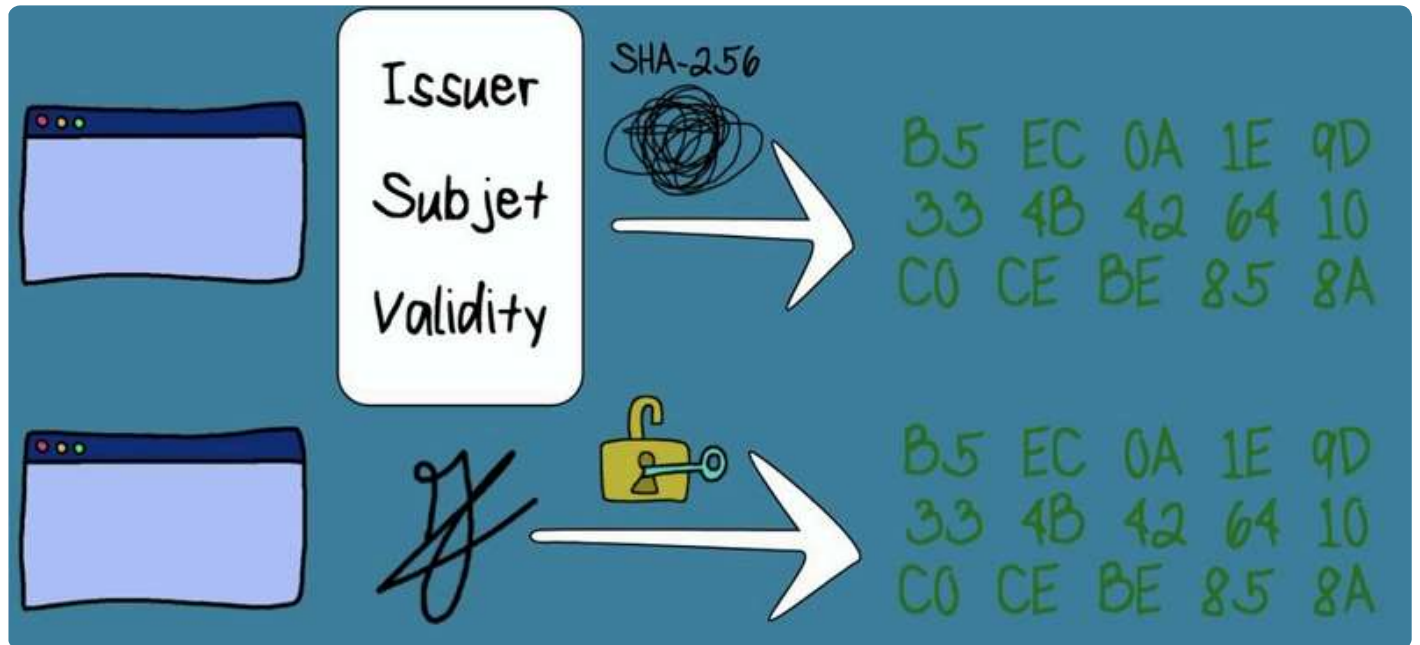
Anyone with the public key of the certificate authority which has signed this certificate can decrypt and verify that the certificate is authentic and has not been tampered.



When the browser receives the certificate, it will hash all the information on its side

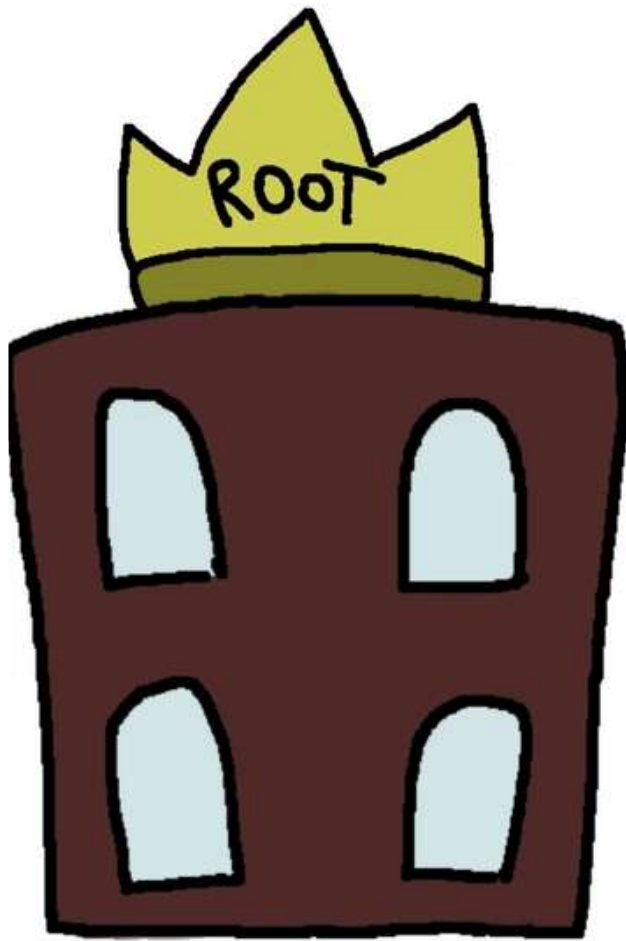
using the same hash algorithm, in this example, SHA-256. Then, it decrypts the signature using the certificate authority's public key to have the hash received from the certificate authority.

If both hashes match, then the browser can confirm that this certificate is truly coming from the claimed certificate authority.

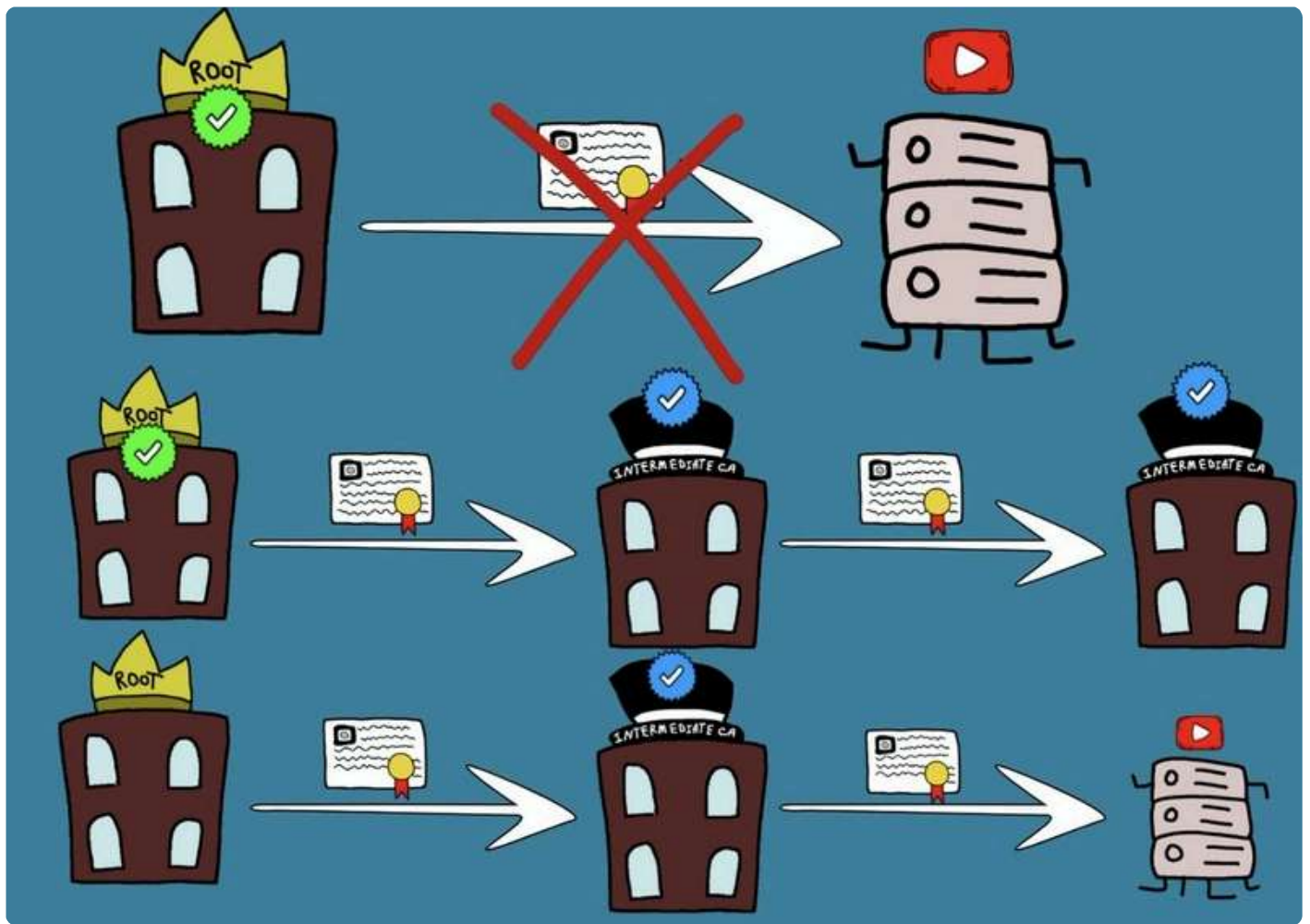


Hierarchies for Certificate Authorities

There exists hierarchies for certificate authorities, the root certificate authority and the intermediate certificate authority.



These root CA does not actually directly issue any digital certificates for servers. It only issues digital certificates for intermediate CAs that act on its behalf. The intermediate CAs can either issue digital certificates for another intermediate CA, or for a server directly.



Hence, There is a chain of trust, from the root CA to the server.

Operating systems come bundled with a trust store, which is a list of trusted root certificate authorities. This list is maintained by the companies that make operating systems such as Apple and Microsoft. They all require a root certificate authority to undergo one or more audits proving their trustworthiness and validity.

How does all of this fit into the picture ? Let's visualize the whole process from the start.

First of all, let's assume youtube.com is a brand new startup. Understanding the need to be secure on a publicly available internet, Youtube would have to own a trusted digital certificate, and present it to its visitors.



Step 1: Youtube goes shopping for a digital certificate

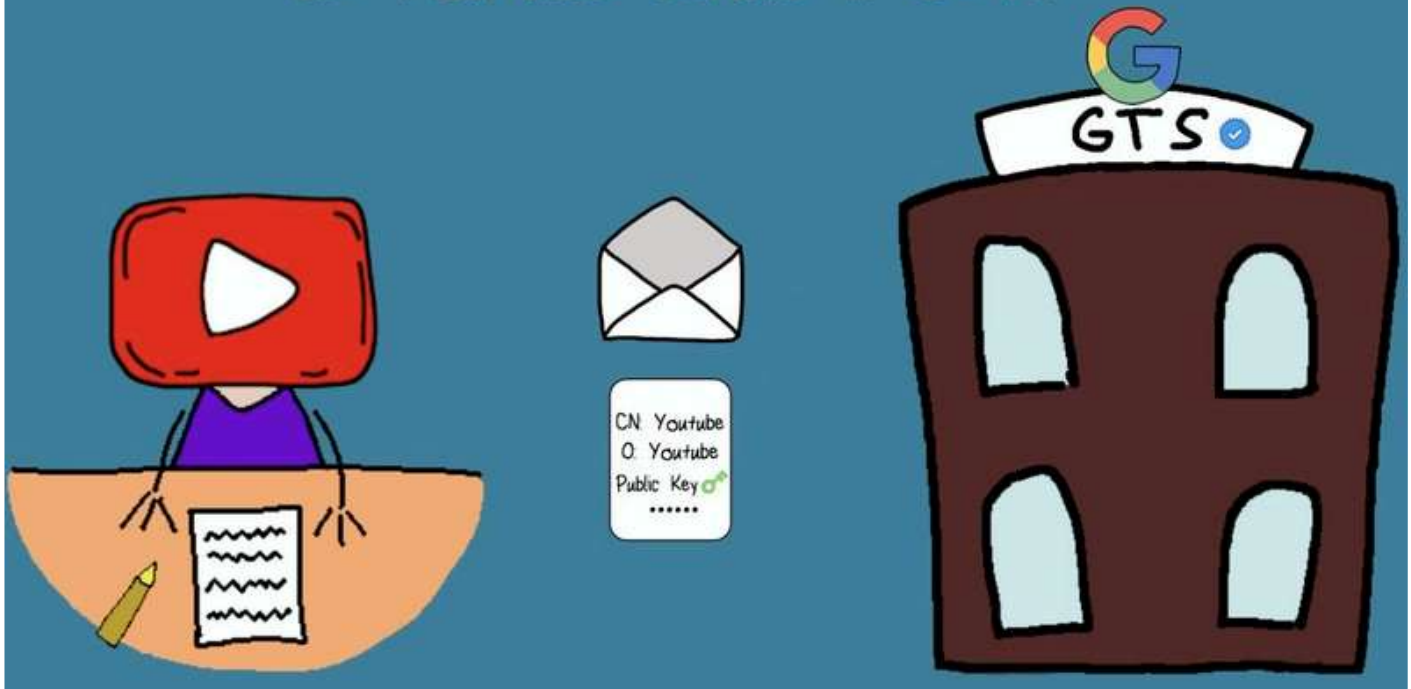
First of all, Youtube goes searching for an intermediate certificate authority. Remember that intermediate certificate authorities are middlemen between servers and root certificate authorities. In the case of Youtube, Google has its own certificate authority called Google Trust Services.

1- Youtube searches for an Intermediate CA

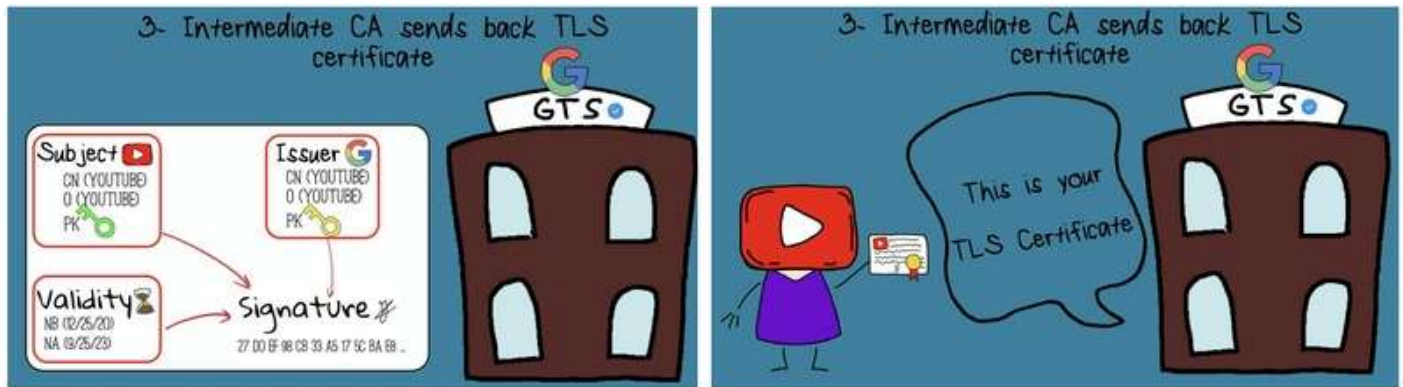


After picking an intermediate CA, Youtube makes a certificate signing request. The certificate signing request will contain information that will be included in Youtube's digital certificate, such as common name, organization, and most importantly the public key of Youtube. Youtube will send the certificate signing request to the intermediate CA.

2- Youtube sends a CSR



The intermediate certificate authority will verify the CSR containing information about the owner also referred to as the subject. Then, it adds field like issuer information, which is information about the intermediate certificate, and the validity, then signs all these information as explained previously in the signing process. After signing, the intermediate certificate authority will send back the digital certificate for youtube.



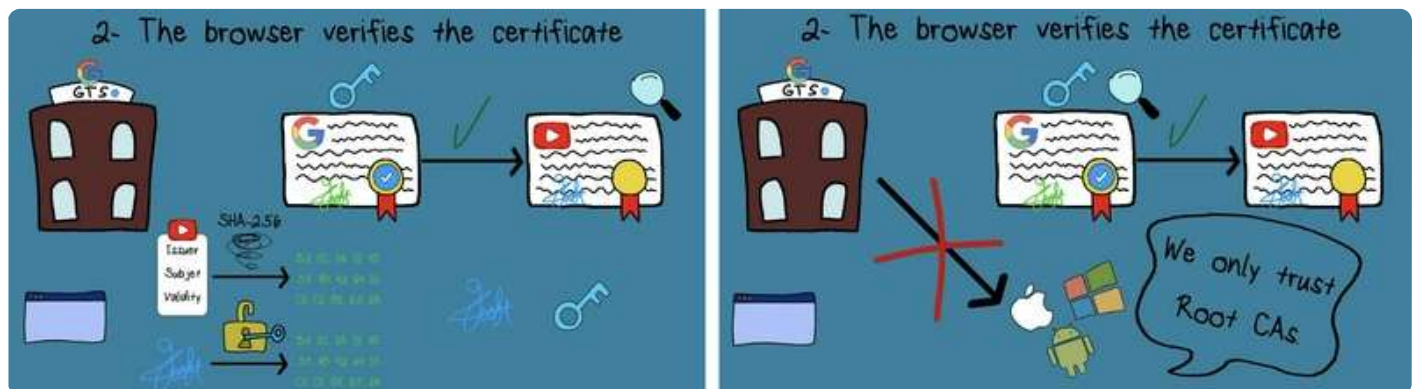
Youtube can now attach its newly bought digital certificate to its web servers.

Step 2: JayP connects to youtube.com to watch fitness videos



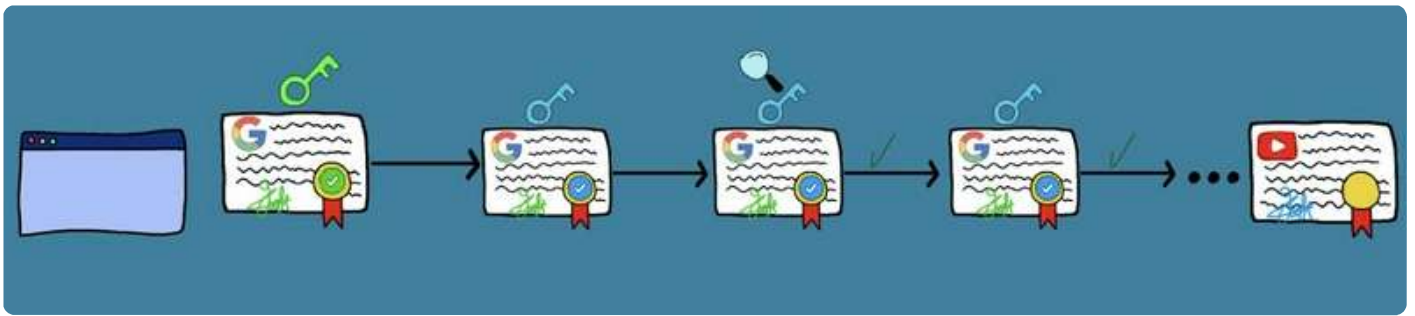
When he will connect to Youtube, Youtube will send its own certificate and certificates of intermediate certificate authorities. The root certificate authority will not be sent because it is available on JayP's Operating System.

Youtube's certificate will include the issuer information, which is the intermediate certificate authority. The browser will hence know that this certificate is signed by Google Trust Services Intermediate Certificate Authority, and will have its certificate. Any digital certificate will contain information, and a signature, as explained previously. The browser will hash all the information to obtain their hash value. Then, it will decrypt the signature using the issuer's public key, in other words, the intermediate certificate authority's public key. Again, remember that the signature can only be decrypted using the issuer's public key. After that the browser will compare the two hash values, if they are the same, then this part of the chain is verified, and the browser will move on verifying the intermediate certificate authority, because as explained, intermediate certificate authorities are not trusted directly by the operating systems.



Okay now the browser will verify the certificate of Google Trust Intermediate Certificate Authority. This intermediate certificate is signed by Google Trust Services Root Certificate authority.

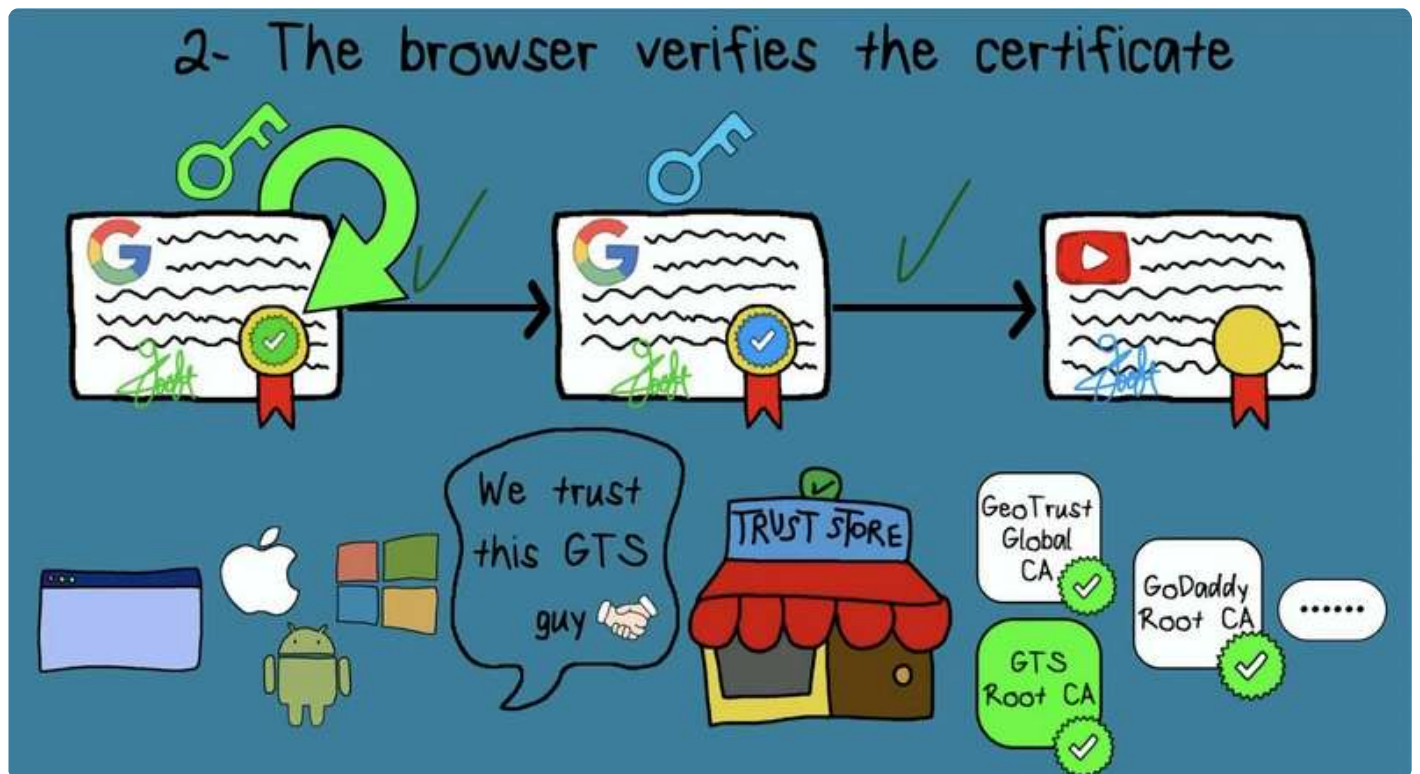
Please note that before this step, we could have many intermediate certificate authorities between the end certificate and the root certificate, and for each part of the chain the same verification process will happen.



In our example, we will just stick to one.

The signature of the certificate of the intermediate certificate authority will be verified with the public key of the root certificate authority, as previously.

When reaching the root certificate authority, that's where the chain ends. The root certificate authority, which is self signed, will be available in JayPee's operating system's trust store.

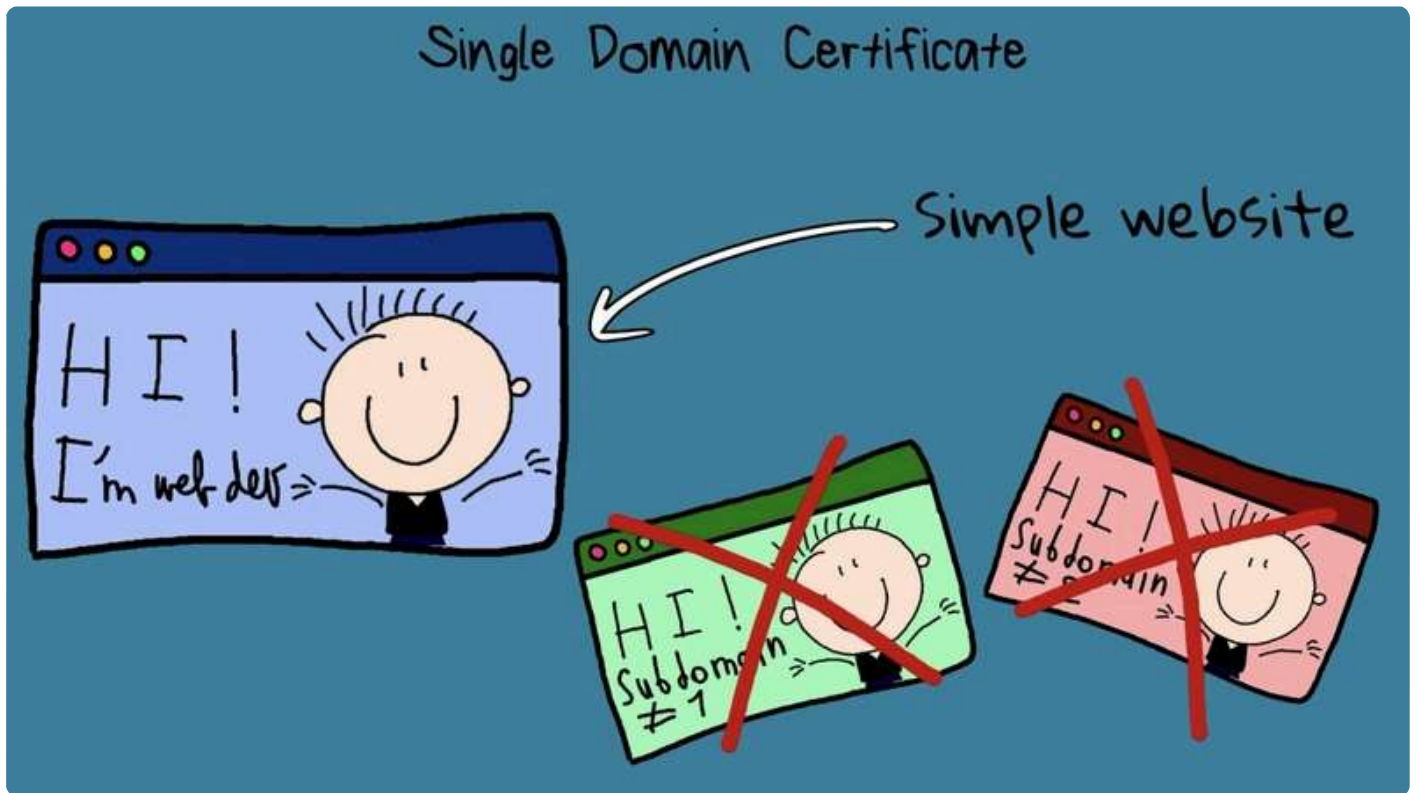


That's it! The browser, after this verification, will display a nice looking lock icon, like the one in your browser. Now it is guaranteed that the communication is really with Youtube, and that it is safe (HOORAY)

Three types of Domain Certificates

Before we wrap up, we will shortly explain three main types of TLS certificates.

1) Single Domain Certificate



A single domain certificate is designed to secure a single fully qualified domain name. It means that the certificate is valid only for one specific domain and does not cover any subdomains or additional domains. For example, for the domain "www.youtube.com," a single domain certificate would secure only that domain and not any other variations like "academy.youtube.com" or "blog.youtube.com"

Use cases: Single domain certificates are ideal when you have a single website or web application that does not require additional subdomains. They are the most straightforward and cost-effective option for securing a specific domain.

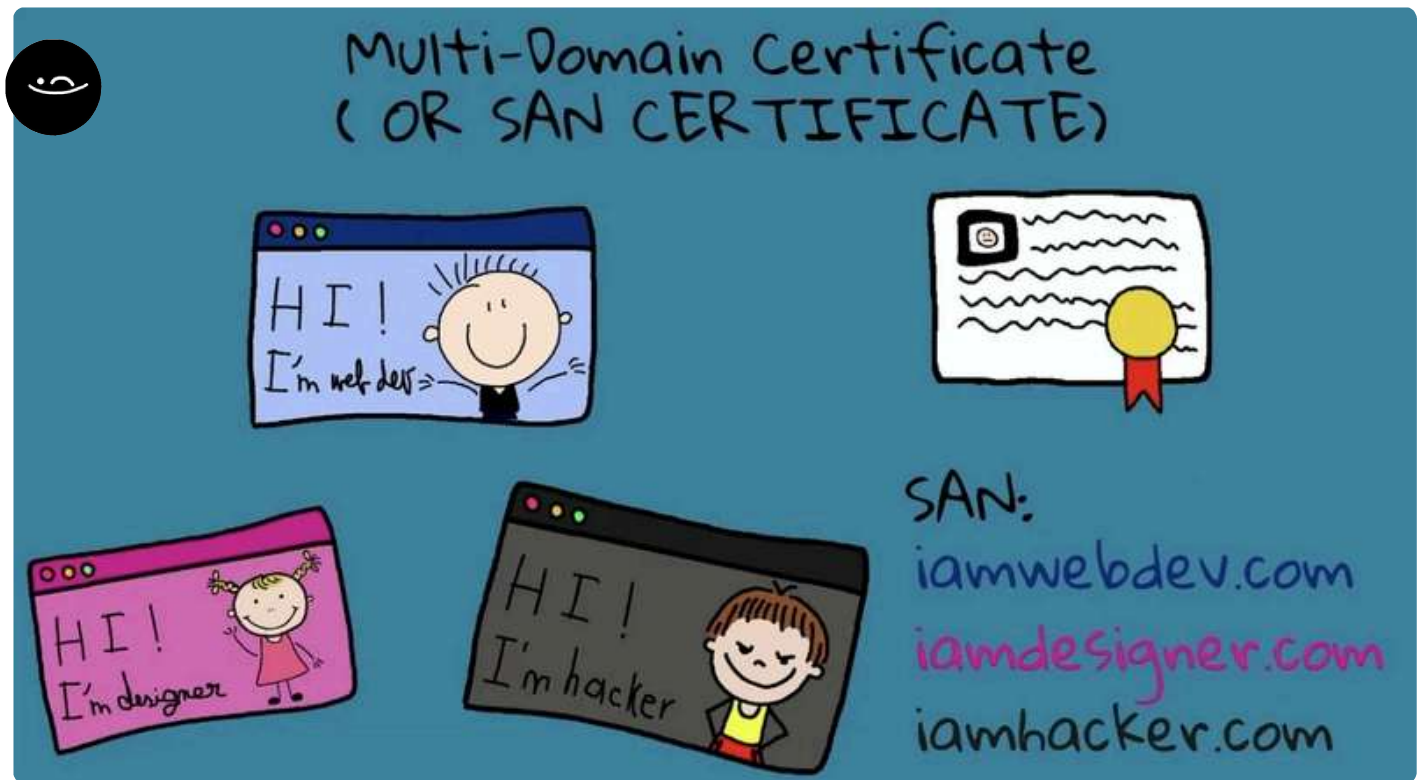
2) Wildcard Certificate



Next we have a wildcard certificate, which is a type of TLS certificate issued for a main domain and all its subdomains using a single certificate. The wildcard character asterisk is used in the Common Name (CN) field or Subject Alternative Name (SAN) field to indicate that any subdomain under the specified domain is covered. For example, if you have a wildcard certificate for ".youtube.com," it would secure "youtube.com," "academy.youtube.com," "blog.youtube.com", and so on.

Use cases: Wildcard certificates saves you from having to manage and renew individual certificates for each subdomain. However, it's essential to note that wildcard certificates only cover one level of subdomains. For instance, a wildcard certificate for "*.youtube.com" would not cover "academy.blog.youtube.com."

3) Multi-Domain (SAN) Certificate



A multi-domain certificate, also known as a Subject Alternative Name certificate, allows you to secure multiple unrelated domains within a single certificate. Each domain or subdomain to be secured is listed in the Subject Alternative Name (SAN) extension of the certificate.

For example, Google and other large companies would use these multi-domain certificates.

One of the main benefits would be Simplified Certificate Management: Google operates a lot of web services and applications, each with its own domain or subdomain. Using multi-domain certificates allows them to consolidate the management of certificates for multiple domains or subdomains into a single certificate. This streamlines the process of certificate issuance, renewal, and monitoring.

In my next article I will discuss encryption with TLS

Top comments (0)